Final

Durum	
Başladı	
Tamamlandı	
Süre	
Not	34,00 üzerinden **18,00** (**52,94**%)

Soru 1

Doğru

1,00 üzerinden 1,00 notunu ver

ArrayList<int> list = new ArrayList<int>(); will have a compile error.

- Doğru
- Yanlış

Doğru cevap 'Doğru'.

Soru 2

Yanlış

1,00 üzerinden 0,00 notunu ver

An abstract class never has concrete methods.

- Doğru
- Yanlış

Doğru cevap 'Yanlış'.

Soru 3

Doğru

1,00 üzerinden 1,00 notunu ver

A Queue is a type of Last In First Out data structure.

- Doğru
- Yanlış

Doğru cevap 'Yanlış'.

Soru 4

Yanlış

1,00 üzerinden 0,00 notunu ver

A constructor is not required to create an object of a class.

- Doğru ✗
- Yanlış

Doğru cevap 'Yanlış'.

Soru 5

Doğru

1,00 üzerinden 1,00 notunu ver

The Object class contains a method called *toString()* that returns a String representation of the Object. Therefore, if we can call the *toString()* method on any object. It is the method called if the object is sent as a parameter to the *println()* method.

- Doğru ✓
- Yanlış

Doğru cevap 'Doğru'.

Soru 6

Kısmen doğru

5,00 üzerinden 3,00 notunu ver

Match the definition with its term. Choose the correct term for the definition.

Two methods with the same name, return type, and parameter list. One exists in a parent class but the other modifies the implementation.	Override ✓
A classlike construct that contains only constants and abstract methods.	Abstract Class ✗
Two methods with the same name and return type but different parameter lists. Can be in the same class.	Overload ✓
Cannot create an Object from this class. Can contain abstract methods that will be defined in subclasses.	Interface ✗
Can create an Object from this class. Cannot contain abstract methods.	Non-Abstract Class ✓

Doğru cevap: Two methods with the same name, return type, and parameter list. One exists in a parent class but the other modifies the implementation. → Override, A classlike construct that contains only constants and abstract methods. → Interface, Two methods with the same name and return type but different parameter lists. Can be in the same class. → Overload, Cannot create an Object from this class. Can contain abstract methods that will be defined in subclasses. → Abstract Class, Can create an Object from this class. Cannot contain abstract methods. → Non-Abstract Class

Soru **bilgi**

I gave two different students the below code and asked each to fill in the blanks with anything that would make the code compile and run without errors. I then ran their code and saw different outputs as described in the following questions. Note: if one of the students created a class as part of the submission, show in the extra box what that code could be.

```
public static void main(String[] args) {

    _______________ refVar1 = _______________;

    _______________ refVar2 = _______________;

    System.out.println(refVar1 == refVar2);
    System.out.println(refVar1.equals(refVar2));
}
```

Soru **7**

Kısmen doğru

8,00 üzerinden 6,00 notunu ver

Student 1's output was as follows:

```
false
true
```

Her code was:

String	✓	refVar1 =	new String("CSE")	✓	;
String	✓	refVar1 =	refVar1	✗	;

Doğru cevap:
Student 1's output was as follows:

```
false
true
```

Her code was:
[String] refVar1 = [new String("CSE")];
[String] refVar1 = [new String("CSE")];

Soru 8

Kısmen doğru

8,00 üzerinden 2,00 notunu ver

Student 1's output was as follows:

```
true
false
```

Her code was:

Integer ✗ refVar1 = new Integer(1) ✗ ;

Integer ✗ refVar2 = refVar1 ✓ ;

Doğru cevap:

Student 1's output was as follows:

```
true
false
```

Her code was:

[MyClass] refVar1 = [new MyClass(1)];

[MyClass] refVar2 = [refVar1];

Soru 9

Tamamlandı

8,00 üzerinden 4,00 notunu ver

If one of the students created a special class for this output, write that class and any necessary methods in the box provided. Do not leave this question blank. If no extra classes are required, you must type "No Extra Classes" in the box below.

No Extra Classes

Soru **bilgi**

Due:

Monday, 2-June-2025 by 10:20

Deliverables:

The following Java file should be submitted to moodle by the due date and time specified above. Submissions received after the deadline will not be accepted.

- Dictionary.java

Specifications:

Overview: Python gives access to a data structure called a dictionary. It operates similarly to an array but instead of integer keys, it can use any datatype to index a value. You will write a java class that operates like a Python dictionary.

Requirements: Write and modify the following classes:

1. Dictionary
    1. Attributes
        1. You will create the attributes that you think are necessary
    2. Methods
        1. Constructor that creates an empty dictionary
            1. The key and value datatypes must be defined (see the examples below)
        2. get(key : type defined by constructor)
            1. Returns the value for this key
            2. Return type is defined by the constructor
            3. If key does not exist in this dictionary, throws a KeyNotFoundException (defined below)
        3. update(key, value) : types defined by the constructor
            1. Returns none
            2. If the key exists in the dictionary, changes the value for that key to the value passed
            3. If the key does not exist in the dictionary, adds the key/value pair to the dictionary
        4. copy()
            1. Returns a new dictionary that is a copy of the key value pairs of this dictionary
        5. keys()
            1. Returns a Collection of the keys for this dictionary
            2. Collection type must be a reasonable type for holding keys
        6. toString()
            1. Returns a string representation of the contents of the dictionary of the form "{K=V, ...}"
2. KeyNotFoundException is a child of RuntimeException
    1. Attributes
        1. You will create the attributes that you think are necessary
    2. Methods
        1. Constructor that takes an Object parameter for the key that was not found
        2. toString()
            1. Returns a String of the form "KeyNotFoundException Key = " + {keyValueView the contents of the coding description. You will create two versions of this project.

Examples:

Code Run	Output

`public class TestDictionary {`
`  public static void main(String[] args) {`
`  Dictionary<String, String> d = new Dictionary<String, String>();`
`  try {`
`  System.out.println(d.get("model"));`
`  } catch (Exception ex) {`
`  System.out.println(ex);`
`  }`
	
	KeyNotFoundException Key = model

`  d.update("brand", "Ford");`
`  d.update("model", "Mustang");`
`  d.update("year", "1964");`
`  System.out.println(d.get("model"));`
`  System.out.println(d.keys());`
`  System.out.println(d);`
`  Dictionary<String, String> d2 = d.copy();`
`  d2.update("year", "1969");`
`  System.out.println(d);`
`  System.out.println(d2);`
	
	Mustang
	[year, model, brand]
	{year:1964, model:Mustang, brand:Ford}
	
	{year:1964, model:Mustang, brand:Ford}
	{year:1969, model:Mustang, brand:Ford}

`  try {`
`  System.out.println(d.get("Model"));`
`  } catch (Exception ex) {`
`  System.out.println(ex);`
`  }`
	
	KeyNotFoundException Key = Model

`  d.clear();`
`  System.out.println(d);`
`  System.out.println(d2);`
	
	{}
	{year:1969, model:Mustang, brand:Ford}

`  Dictionary<String, Integer> myCourse = new Dictionary<String, Integer>();`
`  myCourse.update("Attendance", 20);`
`  myCourse.update("Midterm", 20);`
`  myCourse.update("Homework", 20);`
`  myCourse.update("Final", 40);`
`  System.out.println(myCourse);`
	
	{Attendance:20, Midterm:20, Final:40, Homework:20}

`  Dictionary<Integer, Dictionary> courses = new Dictionary<Integer, Dictionary>();`
`  courses.update(101, myCourse);`
`  courses.update(102, myCourse.copy());`
`  courses.get(102).update("Final", 50);`
`  System.out.println(courses);`
	
	{101:{Attendance:20, Midterm:20, Final:40, Homework:20},
	102:{Attendance:20, Midterm:20, Final:50, Homework:20}}

`  System.out.println(courses.get(204));`
`  }`
`}`
	Exception in thread "main" KeyNotFoundException Key = 204
	`  at Dictionary.get(Dictionary.java:37)`
	`  at TestDictionary.main(TestDictionary.java:39)`

For each, I have already included the available classes you may use for your solution through import statements. These import lists are given below:

Dictionary 1:

```
import java.io.File;
import java.io.FileWriter;
import java.io.PrintWriter;
import java.lang.Exception;
import java.lang.RuntimeException;
import java.util.Collection;
import java.util.HashMap;
import java.util.LinkedHashSet;
import java.util.LinkedList;
import java.util.List;
import java.util.Map;
import java.util.Random;
import java.util.Scanner;
import java.util.Set;
import java.util.TreeMap;

// You may not modify or add any lines of code before this
```

Dictionary 2:

```
import java.io.File;
import java.io.FileWriter;
import java.io.PrintWriter;
import java.lang.Exception;
import java.lang.RuntimeException;
import java.util.Arrays;
import java.util.ArrayList;
import java.util.Collection;
import java.util.HashSet;
import java.util.LinkedList;
import java.util.List;
import java.util.Random;
import java.util.Scanner;
import java.util.Set;
import java.util.TreeSet;

// You may not modify or add any lines of code before this
```